Lesson 13 Teacher's Key: A Hit That Reacts

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing Coroutines - Unity's built-in tool for pausing a function mid-execution using yield return, without freezing the rest of the game - genuinely new this week, and directly motivated by needing exactly this capability for the first time. Also introduces calling StartCoroutine() on a DIFFERENT object's MonoBehaviour than the one detecting the event, so the pause happens on the object actually being delayed.

Level 1 - Full Worked Solution

- Hit detection does not move. Bullet.cs's OnTriggerEnter2D has owned this since Lesson 4 (checking other.CompareTag("Enemy")) and still does - nothing about Lesson 13 changes WHO detects a hit, only what happens to the Enemy afterward.
- HandleHit() lives on Enemy.cs, not Bullet.cs, and is a Coroutine (IEnumerator return type) rather than a plain function - this is required because yield return WaitForSeconds() only works inside a Coroutine. It lives on Enemy specifically because Enemy is the object being delayed; Bullet is destroyed immediately and can't be the one doing the waiting.
- Bullet.cs starts Enemy's Coroutine on Enemy's own instance - enemy.StartCoroutine(enemy.HandleHit()), not StartCoroutine(enemy.HandleHit()) - a Coroutine always runs on whichever MonoBehaviour StartCoroutine() is called on, so calling it on enemy (not on this/Bullet) is what lets the Enemy keep flashing after the Bullet that hit it is already gone.
- hitSound and GameManager.RegisterKill() both stay in Bullet.cs's OnTriggerEnter2D, running immediately, exactly as before - score/sound/wave/win-check happen at exactly the same instant as Lesson 4/7, completely unchanged. Only the Enemy's own Destroy(gameObject), inside its own HandleHit(), moves to after the yield - that is the one and only thing actually delayed.
- other.enabled = false runs in Bullet.cs, disabling the Enemy's Collider2D the instant it's hit, before enemy.StartCoroutine(...) is even called - this is the actual double-hit guard this week. Because the Enemy no longer disappears instantly, it would otherwise sit there, still collidable, for the full 0.1 seconds while flashing - long enough for a second overlapping bullet to trigger a second hit on the exact same enemy.
- hitSound now plays via Camera.main.transform.position rather than Lesson 4's original transform.position - PlayClipAtPoint() always spawns a 3D-blend temporary audio source with real distance rolloff, so a sound played at the enemy's own position can end up nearly inaudible depending on camera distance. Every other round-ending sound in

this course (winSound, waveClip, gameOverSound, lifeLostSound) already used Camera.main.transform.position from the moment it was introduced; hitSound is corrected to match as of this lesson.

Highlighted lines are new this week; everything else is code you already have.

```
// Bullet.cs's OnTriggerEnter2D (revised):
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        other.enabled = false;
        Enemy enemy = other.GetComponent<Enemy>();
        AudioSource.PlayClipAtPoint(hitSound,
Camera.main.transform.position);
        GameManager.RegisterKill();
        enemy.StartCoroutine(enemy.HandleHit());
        Destroy(gameObject);
    }
}

// Enemy.cs (new fields and Coroutine):
public GameObject explosionPrefab;
public SpriteRenderer spriteRenderer;
public Sprite flashSprite;

public IEnumerator HandleHit() {
    Instantiate(explosionPrefab, transform.position, Quaternion.identity);
    spriteRenderer.sprite = flashSprite;
    yield return new WaitForSeconds(0.1f);
    Destroy(gameObject);
}
```



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

Why This Structure

- Coroutines are the right tool here specifically because the pause needs to happen on the Enemy without restructuring Bullet's own hit-detection - a much smaller change than GameMaker or TIC-80 need for the same idea.
- Keeping hit detection on Bullet.cs (rather than moving it to Enemy.cs, which an earlier draft of this lesson mistakenly did) preserves the single, unambiguous rule this course has used since Lesson 4: only Bullet's OnTriggerEnter2D ever calls GameManager.RegisterKill(). Splitting detection and delayed-removal across two objects is fine; splitting WHO detects the hit is not, and would quietly contradict every lesson before this one.
- Giving the explosion prefab its own Destroy(gameObject, 0.3f) call in its own Start() (rather than having the Enemy manage its lifetime) keeps every object responsible for cleaning up only itself - the same discipline bullets and enemies have followed all course.
- The flash swaps sprite rather than tinting spriteRenderer.color white, for the same reason Lesson 2's Recolor swaps sprites instead of tinting them: color multiplies against the sprite's existing color, and multiplying any color by white is a no-op - our enemy sprite is opaque red, so a white color tint would produce zero visible change.
- Moving hitSound's call site to Camera.main.transform.position fixes a real, previously undocumented bug: AudioSource.PlayClipAtPoint() always creates its temporary audio source with 3D spatial blend, so distance from the AudioListener (on the camera) silently

attenuates volume no matter what volumeScale argument is passed - volumeScale is a 0-1 percentage of the clip's own volume, not a boost, so it can never compensate for rolloff. Playing it at the camera's own position, exactly like every other round-ending sound already does, sidesteps the problem entirely.

Level 2 / Level 3 Talking Points

- Ask students to explain why HandleHit() needs to be IEnumerator instead of void - because yield return can only appear inside a function with that return type, a small but important C# rule worth naming explicitly.
- Ask students why HandleHit() is started as enemy.StartCoroutine(enemy.HandleHit()) rather than just StartCoroutine(enemy.HandleHit()) from inside Bullet - because a Coroutine runs on whichever MonoBehaviour StartCoroutine() is called on, and Bullet is about to destroy itself, which would otherwise cancel the Coroutine along with it.
- For students comparing engines: Godot's await and Unity's Coroutine solve the identical problem with similar underlying mechanisms, while GameMaker and TIC-80 need a genuinely different structural approach because their languages don't support pausing mid-script at all.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
// Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
// confirm score/enemies_remaining/wave-clear/win-check all still happen
at exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
// confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Enemy.cs - Level 3 flicker version of HandleHit():
public Sprite normalSprite;

public IEnumerator HandleHit() {
    Instantiate(explosionPrefab, transform.position, Quaternion.identity);
    spriteRenderer.sprite = flashSprite;
    yield return new WaitForSeconds(0.03f);
    spriteRenderer.sprite = normalSprite;
    yield return new WaitForSeconds(0.03f);
    spriteRenderer.sprite = flashSprite;
    yield return new WaitForSeconds(0.04f);
    Destroy(gameObject);
}

// Explosion prefab's own script - Level 3 expanding-scale animation:
void Start() {
    Destroy(gameObject, 0.3f);
}

void Update() {
```

```
    transform.localScale += Vector3.one * Time.deltaTime * 2f;
}
```

Common Student Mistakes

- Writing HandleHit() as a regular void function instead of IEnumerator, which won't compile once yield return is added.
- Calling StartCoroutine(enemy.HandleHit()) from inside Bullet.cs instead of enemy.StartCoroutine(enemy.HandleHit()) - this compiles, but the Coroutine runs on Bullet, which is destroyed moments later, silently canceling it before the yield ever completes.
- Moving hit detection itself onto Enemy's OnTriggerEnter2D instead of keeping it on Bullet's - a tempting-looking shortcut that quietly breaks the "only Bullet calls RegisterKill()" rule this course has enforced since Lesson 4.
- Forgetting other.enabled = false, allowing a second overlapping bullet to also trigger OnTriggerEnter2D on the same, still-flashing enemy and double-register a kill.
- Leaving hitSound's call at transform.position (Lesson 4's original placement) instead of moving it to Camera.main.transform.position - compiles and appears to work, but plays back much quieter than every other sound in the game.